VQ4DiT: Efficient Post-Training Vector Quantization for Diffusion Transformers

Juncan Deng[1*], **Shuaiting Li**[1*], **Zeyu Wang**[1], **Hong Gu**[2], **Kedong Xu**[2], **Kejie Huang**[1]

[1]Zhejiang University, [2]vivo Mobile Communication Co., Ltd
{dengjuncan, list, wangzeyu2020, huangkejie}@zju.edu.cn,
{guhong, xukedong}@vivo.com

arXiv:2408.17131v1 [cs.CV] 30 Aug 2024

Abstract

The Diffusion Transformers Models (DiTs) have transitioned the network architecture from traditional UNets to transformers, demonstrating exceptional capabilities in image generation. Although DiTs have been widely applied to high-definition video generation tasks, their large parameter size hinders inference on edge devices. Vector quantization (VQ) can decompose model weight into a codebook and assignments, allowing extreme weight quantization and significantly reducing memory usage. In this paper, we propose VQ4DiT, a fast post-training vector quantization method for DiTs. We found that traditional VQ methods calibrate only the codebook without calibrating the assignments. This leads to weight sub-vectors being incorrectly assigned to the same assignment, providing inconsistent gradients to the codebook and resulting in a suboptimal result. To address this challenge, VQ4DiT calculates the candidate assignment set for each weight sub-vector based on Euclidean distance and reconstructs the sub-vector based on the weighted average. Then, using the zero-data and block-wise calibration method, the optimal assignment from the set is efficiently selected while calibrating the codebook. VQ4DiT quantizes a DiT XL/2 model on a single NVIDIA A100 GPU within 20 minutes to 5 hours depending on the different quantization settings. Experiments show that VQ4DiT establishes a new state-of-the-art in model size and performance trade-offs, quantizing weights to 2-bit precision while retaining acceptable image generation quality.

1 Introduction

Advancements in pre-trained text-to-image diffusion models (Ho, Jain, and Abbeel 2020; Ho et al. 2022; Ramesh et al. 2022; Rombach et al. 2022; Saharia et al. 2022) have facilitated the successful generation of images that are both complex and highly faithful to the input conditions. Recently, Diffusion Transformers Models (DiTs) (Peebles and Xie 2023) have garnered significant attention due to their superior performance, with OpenAI's SoRA (Brooks et al. 2024) being one of the most prominent applications. DiTs are constructed by sequentially stacking multiple transformer blocks. This architectural design leverages the scaling properties of transformers (Carion et al. 2020; Touvron et al. 2021; Xie et al. 2021; Liu et al. 2021), allowing for more flexible parameter expansion to achieve enhanced performance. Compared to other UNet-based diffusion models, DiTs have demonstrated the ability to generate higher-quality images while having more parameters.

Deploying DiTs can be costly due to their large number of parameters and high computational complexity, which is similar to the challenges encountered with Large Language Models (LLMs). For example, generating a 256×256 resolution image using the DiT XL/2 model can take over 17 seconds and require 10^5 Gflops on an NVIDIA A100 GPU. Moreover, the video generation model SoRA (Brooks et al. 2024), designed concerning DiTs, contains approximately 3 billion parameters. Due to this significant parameter count, deploying them on edge devices with limited computational resources is impractical.

To overcome the deployment challenges, recent research has focused on the efficient deployment of diffusion models, particularly through model quantization (Li et al. 2023a, 2024; He et al. 2024; Wang et al. 2024). Post-training quantization (PTQ) is the most widely used technique because it rapidly quantizes the original model using a small calibration set without requiring multiple iterations of fine-tuning (Yuan et al. 2022; Li et al. 2023a). Meanwhile, vector quantization (VQ) has been shown to compress CNN models to extremely low bit-width (Gersho and Gray 2012; Stock et al. 2019), which could also be advantageous for DiTs. The classic VQ approach maps the weight sub-vectors of each layer to a codebook and assignments using clustering techniques such as the K-Means algorithm (Han, Mao, and Dally 2015), and the codebook is continuously updated.

However, existing quantization methods have several limitations. First, they cannot be directly applied to DiTs, which have different network structures and algorithmic concepts compared to UNet-based diffusion models. Second, PTQ methods significantly reduce model accuracy when quantizing weights to extremely low bit-width (e.g., 2-bit). Third, traditional VQ methods only calibrate the codebook without adjusting the assignments, leading to incorrect assignment of weight sub-vectors, which provides inconsistent gradients to the codebook and ultimately results in suboptimal outcomes.

To overcome these limitations, we introduce a novel post-training vector quantization technique for the extremely low bit-width quantization of DiTs, named VQ4DiT. VQ4DiT

[*]Work done during an internship at vivo Mobile Communication Co., Ltd.

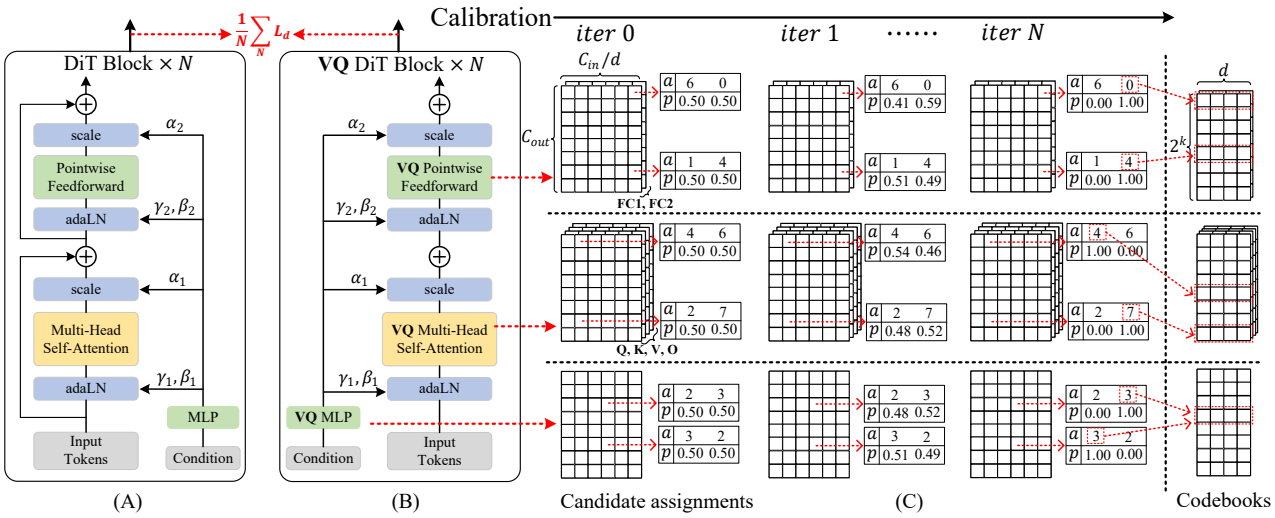

Figure 1: The pipeline of VQ4DiT. (A) DiT blocks. (B) DiT blocks are quantized by vector quantization (VQ). (C) Candidate assignments and codebooks are calibrated by zero-data and block-wise calibration to ultimately obtain the optimal assignments with the highest ratios.

firstly maps the weight sub-vectors of each layer to a codebook using the K-Means algorithm. It then determines a candidate assignment set for each weight sub-vector based on Euclidean distance and reconstructs the sub-vector based on the weighted average. Finally, leveraging the zero-data and block-wise calibration method, the ratio of each candidate assignment is calibrated and the optimal assignment from the set is efficiently selected while simultaneously calibrating the codebook. VQ4DiT ensures that the quantized model achieves results comparable to those of the floating-point model. The contributions are summarized as follows:

- We explore the VQ methods for extremely low bit-width DiTs and introduce DiT-specific improvements for better quantization, which have not been explored in DiT literature.
- We calibrate both the codebook and the assignments of each layer simultaneously, unlike traditional methods that focus solely on codebook calibration.
- Our method achieves competitive evaluation results compared to full-precision models on the ImageNet (Russakovsky et al. 2015) benchmark.

2 Backgrounds and Related Works

2.1 Diffusion Transformer Models

UNet-based diffusion models have garnered significant attention, and research has begun to explore the adoption of transformer architectures (Rombach et al. 2022; Croitoru et al. 2023; Yang et al. 2023) within diffusion models. Recently, Diffusion Transformer Models (DiTs) (Peebles and Xie 2023) have achieved state-of-the-art performance in image generation. Notably, DiTs demonstrate scalability in terms of model size and data representation similar to large language models, making them widely applicable to image and video generation tasks (Brooks et al. 2024; Liu et al. 2024; Zhu et al. 2024).

DiTs consist of N blocks, each containing a Multi-Head Self-Attention (MHSA) and a Pointwise Feedforward (PF) module (Vaswani et al. 2017; Dosovitskiy et al. 2021; Peebles and Xie 2023), both preceded by their respective adaptive Layer Norm (adaLN) (Perez et al. 2018). The structure of the DiT block is illustrated in Figure 1 (A). These blocks sequentially process the noised latent and conditional information, encoded as tokens in a lower-dimensional latent space (Rombach et al. 2022).

In each block, the conditional embedded information $\mathbf{c} \in \mathbb{R}^{d_{in}}$ is converted into scale and shift parameters ($\boldsymbol{\gamma}, \boldsymbol{\beta} \in \mathbb{R}^{d_{in}}$), which are regressed through MLPs and then injected into the noisy latent $\mathbf{z} \in \mathbb{R}^{n \times d_{in}}$ via adaLN:

$$\begin{cases} (\boldsymbol{\gamma}, \boldsymbol{\beta}) = \mathrm{MLP}(\mathbf{c}) \\ \mathrm{adaLN}(\mathbf{z}) = \mathrm{LN}(\mathbf{z}) \odot (\mathbf{1} + \boldsymbol{\gamma}) + \boldsymbol{\beta}, \end{cases} \tag{1}$$

where $\mathrm{LN}(\cdot)$ denotes the Layer Norm (Ba, Kiros, and Hinton 2016). These adaLN modules dynamically adjust the layer normalization before each MHSA and PF module, enhancing DiTs' adaptability to varying conditions and improving the generation quality.

Despite their effectiveness, DiTs demand substantial computational resources to generate high-quality images, which poses challenges to their deployment on edge devices. In this paper, we propose an extremely low bit-width quantization method for DiTs that significantly reduces both time and memory consumption, without the need for a calibration dataset.

2.2 Model Quantization

Let $W \in \mathbb{R}^{o \times i}$ denote the weight, where o represents the output channel and i denotes the input channel. A standard symmetric uniform quantizer approximates the original

Method	$k \times d$	C (MB)	A (MB)	MSE $\downarrow$
FP	n/a	n/a	n/a	0.00
3-bit UQ	n/a	n/a	n/a	7.36e-3
2-bit UQ	n/a	n/a	n/a	3.09e-2
3-bit VQ	64×2	0.10	239.20	1.39e-3
	4096×4	12.25	239.20	1.07e-3
2-bit VQ	256×4	0.77	159.47	3.98e-3
	4096×6	18.38	159.47	3.35e-3

Table 1: Metrics of classic uniform quantization (UQ) and vector quantization (VQ) in the DiT XL/2 256×256 Model. The dimensions of the codebook for VQ are represented as $k \times d$. C(MB) and A(MB) denote the memory usage of all codebooks and all assignments, respectively. 'MSE' denotes the mean square error between floating-point weights and quantized weights.

Method	Fine-tune	FID $\downarrow$	IS $\uparrow$	Precision $\uparrow$
FP	n/a	6.72	243.90	0.7848
3-bit UQ	Yes	1.3e2	9.92	0.1704
2-bit UQ	Yes	2.5e2	2.14	0.1081
3-bit VQ	No	46.40	51.86	0.4756
	Yes	35.14	60.02	0.5979
2-bit VQ	No	86.82	18.12	0.3252
	Yes	66.01	29.48	0.4533

Table 2: Results of classic uniform quantization (UQ) and vector quantization (VQ) in the DiT XL/2 256×256 Model. 'Fine-tune' denotes whether the quantization parameters (e.g., scales or codebooks) are fine-tuned while updating the biases and normalization layers. The timesteps are set to 50 and the classifier-free guidance (CFG) is set to 1.5. The number of generated images is 10000.

floating-point weight W as $\widehat{W} \approx sW_{int}$, where each element in W_{int} is a b-bit integer value and s is a high-precision quantization scale shared across all elements of W.

Uniform quantization and its variants of transformer blocks have been extensively studied, with most of the research focusing on the efficient quantization of model weights to reduce memory overhead. RepQ-ViT (Li et al. 2023b) adopts scale reparameterization to minimize the quantization error. GPTQ (Frantar et al. 2022) compensates for unquantized weights based on Hessian information, achieving a good 4-bit quantization performance. Meanwhile, AWQ (Lin et al. 2023) introduces activation-aware weight quantization, specifically designed to minimize the quantization error in salient weights. Q-DiT (Chen et al. 2024) employs group-wise quantization and utilizes an evolutionary search algorithm to optimize the grouping strategy. However, uniform quantization incurs a larger error at extremely low bit-width quantization due to its limitation of reconstructing weights in equidistant distributions.

A more flexible quantization approach is vector quantization (VQ) (Gersho and Gray 2012; Stock et al. 2019), which expresses W in terms of assignments A and a codebook C. First, VQ divides W into row sub-vectors $w_{i,j} \in \mathbb{R}^{1\times d}$:

$$W = \begin{bmatrix} w_{1,1} & w_{1,2} & \cdots & w_{1,i/d} \\ w_{2,1} & w_{2,2} & \cdots & w_{2,i/d} \\ \vdots & \vdots & \ddots & \vdots \\ w_{o,1} & w_{o,2} & \cdots & w_{o,i/d} \end{bmatrix}, \tag{2}$$

where $o \cdot i/d$ is the total number of sub-vectors. These sub-vectors are quantized to a codebook $C = \{c(1), \ldots, c(k)\} \subseteq \mathbb{R}^{d\times 1}$, where $c(k)$ is referred to as the k-th codeword. The assignments $A = \{a_{i,j} \in \{1, \ldots, k\}\}$ are the indices of each codeword in the codebook that best reconstruct every sub-vectors $\{w_{i,j}\}$. The quantized weight $\widehat{W}$ is reconstructed by replacing each $w_{i,j}$ with $c(a_{i,j})$:

$$\widehat{W} = C[A] = \begin{bmatrix} c(a_{1,1}) & c(a_{1,2}) & \cdots & c(a_{1,i/d}) \\ c(a_{2,1}) & c(a_{2,2}) & \cdots & c(a_{2,i/d}) \\ \vdots & \vdots & \ddots & \vdots \\ c(a_{o,1}) & c(a_{o,2}) & \cdots & c(a_{o,i/d}) \end{bmatrix}. \tag{3}$$

All assignments can be stored using $\frac{o\times i}{d} \times \log_2 k$ bits and the codebook can be stored using $k \times d \times 32$ bits. To the best of our knowledge, no existing research has applied VQ to DiTs.

3 Challenges of Vector Quantization for DiTs

3.1 Trade-off of codebook size

As illustrated in Table 1, we apply the classic uniform quantization (UQ) and vector quantization (VQ) to the DiT XL/2 model. At the same bit-width, VQ results in a much smaller quantization error compared to UQ. The number of codewords k and their dimension d significantly impact both the memory usage of VQ and the quantization error of the weights. Increasing k and d, while keeping the memory usage of assignments constant, reduces the quantization error. However, this also increases the memory usage of the codebook, which is particularly problematic in per-layer VQ. Additionally, increasing k and d prolongs the runtime of the clustering algorithm and increases the subsequent calibration times. These factors necessitate a careful trade-off between quantization error and codebook size.

We utilize $k = 256$ and $d = 4$ for 2-bit quantization, $k = 64$ and $d = 2$ for 3-bit quantization. The memory usage of the codebooks is negligible when compared to the memory requirements for assignments.

3.2 Setups of codebooks and assignments

There are various methods to achieve VQ, one popular method being the K-Means algorithm (Han, Mao, and Dally 2015). However, the quantization error of the weights can

significantly degrade model performance. To mitigate the negative impact, some studies (Martinez et al. 2021; Stock et al. 2019) assume that assignments are sufficiently accurate and use the training set to fine-tune the codebook of each layer. These approaches have yielded good results on smaller CNN networks, such as ResNet18 (He et al. 2016) and VGG16 (Simonyan and Zisserman 2014), with performance close to that of the original models. Unfortunately, fine-tuning quantized DiTs on the ImageNet dataset is time-consuming and computationally intensive, while the accumulation of quantization errors is more pronounced in these large-scale models.

As shown in Table 2, we apply the classic UQ and VQ to the DiT XL/2 model and fine-tune the quantization parameters, ensuring identical quantization settings and updating iterations. Although VQ outperforms UQ, it still falls short of being acceptable at extremely low-width. Moreover, fine-tuning the codebook of each layer only slightly improved the results. The primary reason is that sub-vectors with the same assignment may have gradients pointing in different directions, and the accumulation of these gradients hinders the correct updating of the codeword, which results in sub-optimal codewords.

Our approach differs from previous VQ methods in that we efficiently calibrate both the codebooks and the assignments simultaneously. This strategy allows us to avoid the accumulation of errors in the gradients of the codewords and to achieve better performance in DiTs.

4 VQ4DiT

To address the identified challenges, we propose a novel method for efficiently and accurately vector quantizing DiTs, named Efficient Post-Training Vector Quantization for Diffusion Transformers (VQ4DiT). The description of VQ4DiT is visualized in Figure 1 (B) and (C). In Section 4.1, we decompose the weights of each layer of the model into a codebook and candidate assignment sets, initializing each candidate assignment with an equal ratio. In Section 4.2, we introduce a zero-data and block-wise calibration strategy to calibrate codebooks and candidate assignment sets, ultimately selecting the optimal assignments with the highest ratios.

4.1 Initialization of Codebooks and Candidate Assignment Sets

As shown in Equation 3, the codebook C and assignments A of each layer can be optimized by minimizing the following objective function:

$$\|W - C[A]\|_2^2 = \sum_{o,i/d} \left\| w_{o,i/d} - c(a_{o,i/d}) \right\|_2^2, \quad (4)$$

which can be efficiently minimized by the K-Means algorithm. However, Table 2 demonstrates that the strategy of fine-tuning only the codebook is not effective for DiTs. Our approach considers how to calibrate both the codebook and the assignments simultaneously.

For each weight sub-vector, we calculate its Euclidean distance to all codewords, obtaining the indices of the top n closest codewords:

$$A_c = \{a_{o,i/d}\}_n = \arg\min_k^n \|w_{o,i/d} - c(k)\|_2^2, \quad (5)$$

where A_c is the candidate assignment sets of the sub-vectors and n is the length of each A_c. We assume that each set contains the optimal assignment for each sub-vector, which needs to be determined. To achieve this, we assign softmax ratios R to all members of the set:

$$R = \{r_{o,i/d}\}_n = \{\frac{e^{z_n}}{\sum_{j=1}^{n} e^{z_j}}\}_n, \sum_{o,i/d} \{r_{o,i/d}\}_n = 1, \quad (6)$$

where z_n is the actual value of each ratio. Each ratio of $\{r_{o,i/d}\}_n$ is initialized to $\frac{1}{n}$ and calibrated in the next process. Therefore, $\widehat{W}$ can be reconstructed based on the weighted average, expressed as the formula:

$$RC[A_c] = \begin{bmatrix} r_{1,1}c(\{a_{1,1}\}_n) & \cdots & r_{1,i/d}c(\{a_{1,i/d}\}_n) \\ r_{2,1}c(\{a_{2,1}\}_n) & \cdots & r_{2,i/d}c(\{a_{2,i/d}\}_n) \\ \vdots & \ddots & \vdots \\ r_{o,1}c(\{a_{o,1}\}_n) & \cdots & r_{o,i/d}c(\{a_{o,i/d}\}_n) \end{bmatrix} \quad (7)$$

4.2 Zero-data and block-wise Calibration

Training DiTs typically relies on the ImageNet dataset (Russakovsky et al. 2015). Due to its large number of images and substantial memory usage, calibrating quantized models using this dataset poses significant challenges. To more efficiently quantize DiTs, we propose a zero-data and block-wise calibration strategy, which aligns the performance of quantized models with that of floating-point models without requiring a calibration set.

Specifically, given the same input to both the floating-point model and the quantized model, the mean square error between the outputs of each DiT block at each timestep is computed to calibrate the codebook and the ratios of the candidate assignments for each layer. It is important to note that the input for the initial timestep is Gaussian noise $\epsilon \sim \mathcal{N}(0, \mathbf{I})$, and the inputs for subsequent timesteps are the outputs of the floating-point model from the previous timestep. This ensures that the quantized model does not suffer from calibration collapse due to cumulative quantization errors and that the output styles of the quantized model remain similar. Given the latent code $\mathbf{z}$ of an image and its paired conditional information $\mathbf{y} \in \{1, \ldots, 1000\}$, the block-wise calibration function is computed as:

$$\mathcal{L}_d = \mathbb{E}_{\mathbf{z},\mathbf{y},d,t} \left[\sum_l \left\| d_f^l(\mathbf{z}_t, \mathbf{y}, t, W) - d_q^l(\mathbf{z}_t, \mathbf{y}, t, \widehat{W}) \right\|_2^2 \right] \quad (8)$$

where $\mathbf{z}_t$ represents a noisy latent at timestep $t \sim \text{Uniform}(1, T)$, and $d_f p^l(\circ)$ and $d_q^l(\circ)$ represent the l-th DiT block from the floating-point model and the quantized model, respectively.

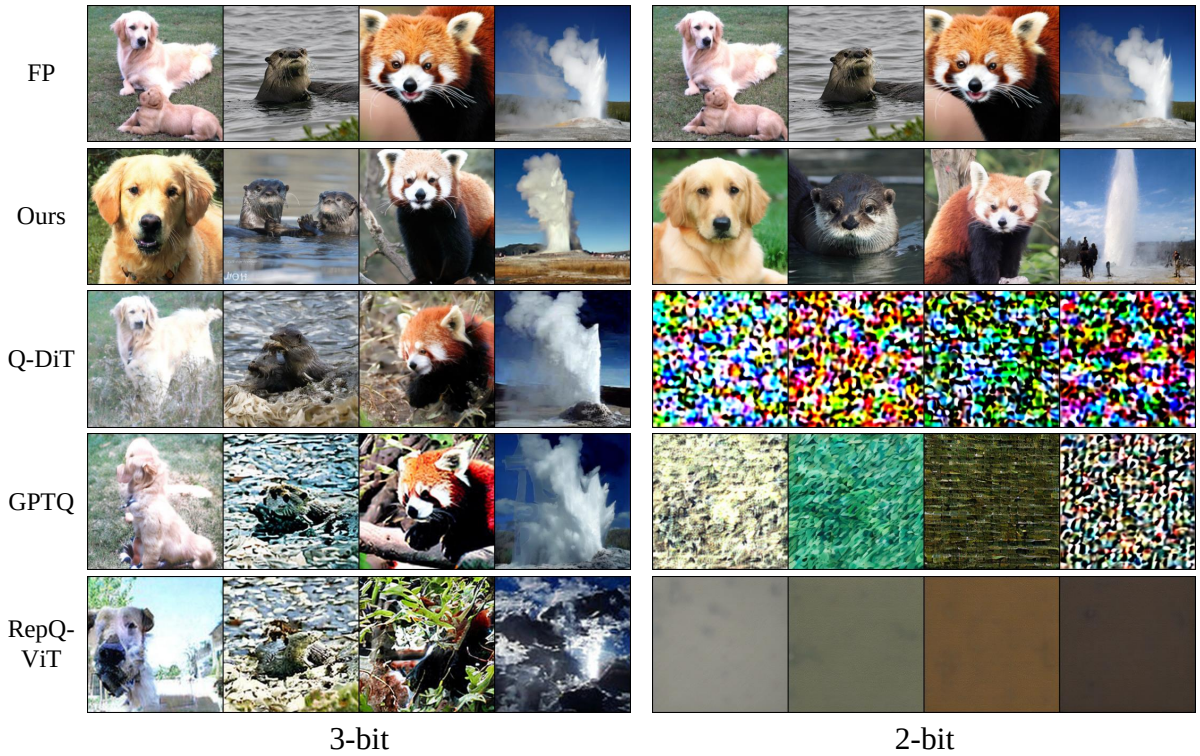

Figure 2: Images generated by VQ4DiT and three strong baselines: RepQ-ViT (Li et al. 2023b), Q-DiT (Chen et al. 2024), and GPTQ (Frantar et al. 2022), with 3-bit and 2-bit quantization on ImageNet 256×256. Our VQ4DiT model is capable of generating high-quality images even at extremely low bit-width.

To accelerate the search for optimal assignments, we augment the mean objective function concerning R as follows:

$$\mathcal{L}_r = \sum_{o,i/d,n} (1 - \left|2 \times \{r_{o,i/d}\}_n - 1\right|)/(\frac{o \times i}{d}). \quad (9)$$

Thus, the final objective function $\mathcal{L}$ is represented as:

$$\mathcal{L} = \lambda_d \mathcal{L}_d + \lambda_r \mathcal{L}_r, \quad (10)$$

where λ_d and λ_r are hyperparameters, both set to 1 for simplicity. During the calibration process, we update the codebooks and ratios through gradient:

$$C \leftarrow C - u\left(\frac{\partial \mathcal{L}}{\partial c}, \theta\right), R \leftarrow R - u\left(\frac{\partial \mathcal{L}}{\partial r}, \theta\right), \quad (11)$$

where $u(\cdot, \cdot)$ is an optimizer with hyperparameters θ. When $\mathcal{L}_r$ falls below a threshold λ (e.g., 10^{-4}), the optimal assignment for each sub-vector is the one with the highest ratio in candidate assignments, after which R is no longer updated.

5 EXPERIMENTS

5.1 Experimental Settings

Models and quantization. The validation setup is generally consistent with the settings used in the original DiT paper (Peebles and Xie 2023). We select the pre-trained DiT XL/2 model as the floating-point reference model, which has two versions for generating images with resolutions of 256×256 and 512×512, respectively. We calibrate all quantized models using RMSprop optimizer, with a constant learning rate of 5×10^{-2} for ratios of candidate assignments and 1×10^{-4} for other parameters. The batch size and iteration are set to 16 and 500 respectively, allowing the experiments to be conducted on a single NVIDIA A100 GPU within 20 minutes to 5 hours. We employ a DDPM scheduler with sampling timesteps of 50, 100, and 250. The classifier-free guidance (CFG) is set to 1.5. To maintain consistency with other baselines, we only quantize the DiT blocks, which are the most computationally intensive components of the DiTs. The length of each candidate assignment set n in our VQ4DiT is 2.

Metrics. To evaluate the quality of generated images, we follow the DiT paper and employed four metrics: Fréchet Inception Distance (FID) (Heusel et al. 2017), spatial FID (sFID) (Salimans et al. 2016; Nash et al. 2021), Inception Score (IS) (Salimans et al. 2016; Barratt and Sharma 2018), and Precision. All metrics were computed using ADM's TensorFlow evaluation toolkit (Dhariwal and Nichol 2021). For both ImageNet 256×256 and ImageNet 512×512, we sample 10k images for evaluation.

Baselines. We compare VQ4DiT with three strong baselines: RepQ-ViT (Li et al. 2023b), GPTQ (Frantar et al. 2022), and Q-DiT (Chen et al. 2024), which are advanced post-training quantization techniques for ViTs, LLMs, and

Timesteps	bit-width	Method	Size (MB)	FID ↓	sFID ↓	IS ↑	Precision ↑
250	32	FP	2553.35	5.33	17.85	275.13	0.8216
	3	RepQ-ViT	239.43	1.5e2	1.3e2	6.73	0.0481
		GPTQ	252.81	50.94	38.60	46.37	0.3932
		Q-DiT	249.67	1.3e2	95.86	12.75	0.1451
		Ours	241.14	**10.59**	**23.03**	**267.46**	**0.9094**
	2	RepQ-ViT	159.64	3.1e2	2.1e2	1.26	0.0002
		GPTQ	172.86	2.8e2	1.4e2	3.97	0.0326
		Q-DiT	168.36	3.1e2	2.1e2	1.28	0.0001
		Ours	162.08	**11.87**	**23.27**	**219.33**	**0.8882**
100	32	FP	2553.35	5.59	18.63	269.67	0.8156
	3	RepQ-ViT	239.43	1.6e2	1.5e2	4.12	0.0429
		GPTQ	252.81	58.94	42.80	40.11	0.3529
		Q-DiT	249.67	1.3e2	1.0e2	11.52	0.1444
		Ours	241.14	**10.74**	**23.90**	**265.63**	**0.9018**
	2	RepQ-ViT	159.64	3.1e2	2.1e2	1.26	0.0003
		GPTQ	172.86	2.7e2	1.3e2	4.17	0.0001
		Q-DiT	168.36	3.1e2	2.1e2	1.28	0.0003
		Ours	162.08	**11.85**	**23.64**	**213.77**	**0.8836**
50	32	FP	2553.35	6.72	21.13	243.90	0.7848
	3	RepQ-ViT	239.43	1.7e2	1.5e2	3.42	0.0.0371
		GPTQ	252.81	71.42	55.39	32.27	0.3256
		Q-DiT	249.67	1.5e2	1.2e2	10.13	0.1265
		Ours	241.14	**11.91**	**24.18**	**263.93**	**0.9096**
	2	RepQ-ViT	159.64	3.1e2	2.1e2	1.26	0.0003
		GPTQ	172.86	3.1e2	1.5e2	4.33	0.0006
		Q-DiT	168.36	3.1e2	2.1e2	1.28	0.0005
		Ours	162.08	**12.42**	**25.16**	**209.95**	**0.8725**

Table 3: Performance comparison on ImageNet 256×256. 'Timesteps' denotes the sampling step of DiTs. 'bit-width' indicates the precision of quantized weights.

Timesteps	bit-width	Method	Size (MB)	FID ↓	sFID ↓	IS ↑	Precision ↑
100	32	FP	2553.35	5.00	19.02	274.78	0.8149
	3	GPTQ	252.81	78.61	40.75	29.69	0.3604
		Q-DiT	249.67	2.0e2	1.2e3	5.32	0.0362
		Ours	241.14	**33.56**	**40.95**	**67.15**	**0.7909**
	2	GPTQ	172.86	3.1e2	1.7e2	2.66	0.0179
		Q-DiT	168.36	3.8e2	2.2e2	1.25	0.0001
		Ours	162.08	**34.32**	**51.08**	**57.03**	**0.7929**
50	32	FP	2553.35	6.02	21.77	246.24	0.7812
	3	GPTQ	252.81	94.35	52.27	23.96	0.3098
		Q-DiT	249.67	2.0e2	1.3e2	4.83	0.0317
		Ours	241.14	**34.57**	**40.31**	**66.61**	**0.7738**
	2	GPTQ	172.86	3.2e2	1.8e2	2.65	0.0170
		Q-DiT	168.36	3.8e2	2.2e2	1.24	0.0.0001
		Ours	162.08	**35.08**	**48.81**	**56.82**	**0.7744**

Table 4: Performance comparison on ImageNet 512×512. 'Timesteps' denotes the sampling step of DiTs. 'bit-width' indicates the precision of quantized weights.

Steps	n	FID ↓	sFID ↓	IS ↑	Precision ↑
50	1	60.16	55.77	32.48	0.5709
	2	12.42	25.16	209.95	0.8725
	3	**12.22**	**24.94**	**214.11**	**0.8802**
	4	14.06	25.12	176.69	0.8750

Table 5: Ablation study on ImageNet 256×256 with 2-bit quantization. n denotes the length of the candidate assignment set.

DiTs, respectively. Considering the structural similarity (Dosovitskiy et al. 2021) between DiTs and the other two types of models, we re-implemented these methods and applied them to DiTs.

5.2 Main Results

Tables 3 and 4 show the quantization results of the DiT XL/2 model on the ImageNet 256×256 and 512×512 datasets using different sample timesteps and weight bit-widths. At a resolution of 256×256, our VQ4DiT achieves performance closest to that of the FP model compared to other methods. Specifically, RepQ-ViT, GPTQ, and Q-DiT undergo a significant performance drop under 3-bit quantization, which worsens as the number of timesteps decreases. In contrast, the FID increases for VQ4DiT by less than 5.3, and the IS decreases by less than 7.7. The metrics of VQ4DiT are very close to those of the FP model, indicating that our method approaches lossless 3-bit compression.

When the bit-width is reduced to 2, the other three algorithms completely collapse. VQ-DiT significantly outperforms the other three methods, with its precision decreasing by only 0.012 compared to 3-bit quantization. Figure 2 shows the generated images by each algorithm, highlighting VQ4DiT's ability to generate high-quality images even at extremely low bit-widths.

Moreover, the validation results at a resolution of 512×512 mirror those at 256×256, with our VQ4DiT consistently demonstrating the best performance. This indicates that VQ4DiT can generate high-quality and high-resolution images with minimal memory usage, which is crucial for deploying DiTs on edge devices.

5.3 Ablation Study

To verify the efficacy of our algorithm, we conduct an ablation study on the challenging 2-bit quantization. In Table 5, we evaluate the different lengths of candidate assignment sets. The detailed result indicates that as n increases from 1 to 2, the performance progressively improves, validating the effectiveness of the assignment calibration. Notably, when $n = 3$, the model demonstrates the most significant performance gain, reducing the FID by 47.97 and the sFID by 30.83. However, as n increases to 4, the performance worsens, suggesting that excessive candidate assignments negatively impact calibration convergence.

To assess whether the optimal assignments yield more accurate gradients for the codebook, We calculate the gradients of the sub-vectors associated with each codeword without calibrating the codebook of each layer. As illustrated in

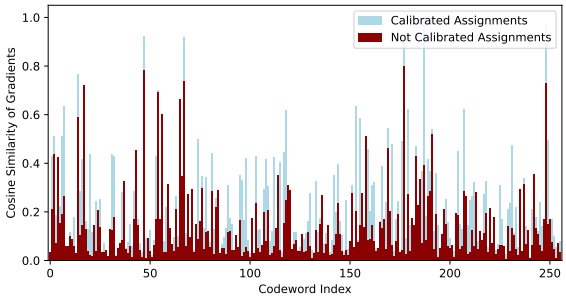

Figure 3: Cosine similarity of gradients of sub-vectors with the same assignment under the two scenarios of whether the assignments are calibrated.

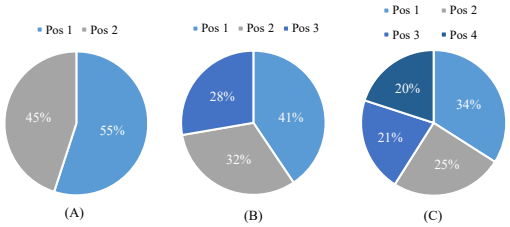

Figure 4: The proportion of position of optimal assignments in the candidate assignment sets with different lengths n. (A) $n = 2$. (B) $n = 3$. (C) $n = 4$.

Figure 3, the cosine similarity of gradients of sub-vectors with the same assignment increases significantly after the assignments are calibrated. This suggests that sub-vectors sharing the same original assignment may produce conflicting gradients for the corresponding codeword, resulting in inaccurate updates. In contrast, our assignment calibration mitigates this issue. Furthermore, as shown in Figure 4, we illustrate the distribution of the optimal assignments. Among the pool of candidate assignments, those with smaller Euclidean distances to the sub-vectors are more likely to be selected as optimal assignments.

6 Conclusion

In this paper, we propose a novel post-training vector quantization method, VQ4DiT, for the efficient quantization of Diffusion Transformers Models (DiTs). Our analysis identifies two main challenges when applying vector quantization (VQ) to DiTs: the need to balance the codebook size with quantization error, and the possibility that different sub-vectors with the same assignment might provide inconsistent gradient directions to the codeword. To address these challenges, we first calculate a candidate assignment set for each sub-vector. We then design a zero-data and block-wise calibration process to progressively calibrate each layer's codebook and candidate assignment sets, ultimately leading to optimal assignments and codebooks. Experimental results demonstrate that our VQ4DiT method effectively quantizes DiT weights to 2-bit precision while maintaining high-quality image generation capabilities.

References

Ba, J. L.; Kiros, J. R.; and Hinton, G. E. 2016. Layer normalization. *arXiv preprint arXiv:1607.06450*.

Barratt, S.; and Sharma, R. 2018. A note on the inception score. *arXiv preprint arXiv:1801.01973*.

Brooks, T.; Peebles, B.; Holmes, C.; DePue, W.; Guo, Y.; Jing, L.; Schnurr, D.; Taylor, J.; Luhman, T.; Luhman, E.; Ng, C.; Wang, R.; and Ramesh, A. 2024. Video generation models as world simulators. *arXiv preprint arXiv:2402.17177*.

Carion, N.; Massa, F.; Synnaeve, G.; Usunier, N.; Kirillov, A.; and Zagoruyko, S. 2020. End-to-end object detection with transformers. In *ECCV*.

Chen, L.; Meng, Y.; Tang, C.; Ma, X.; Jiang, J.; Wang, X.; Wang, Z.; and Zhu, W. 2024. Q-DiT: Accurate Post-Training Quantization for Diffusion Transformers. *arXiv preprint arXiv:2406.17343*.

Croitoru, F.-A.; Hondru, V.; Ionescu, R. T.; and Shah, M. 2023. Diffusion models in vision: A survey. *IEEE TPAMI*.

Dhariwal, P.; and Nichol, A. 2021. Diffusion models beat gans on image synthesis. *Advances in neural information processing systems*, 34: 8780–8794.

Dosovitskiy, A.; Beyer, L.; Kolesnikov, A.; Weissenborn, D.; Zhai, X.; Unterthiner, T.; Dehghani, M.; Minderer, M.; Heigold, G.; Gelly, S.; et al. 2021. An image is worth 16x16 words: Transformers for image recognition at scale. In *ICLR*.

Frantar, E.; Ashkboos, S.; Hoefler, T.; and Alistarh, D. 2022. Gptq: Accurate post-training quantization for generative pre-trained transformers. *arXiv preprint arXiv:2210.17323*.

Gersho, A.; and Gray, R. M. 2012. *Vector quantization and signal compression*, volume 159. Springer Science & Business Media.

Han, S.; Mao, H.; and Dally, W. J. 2015. Deep compression: Compressing deep neural networks with pruning, trained quantization and huffman coding. *arXiv preprint arXiv:1510.00149*.

He, K.; Zhang, X.; Ren, S.; and Sun, J. 2016. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, 770–778.

He, Y.; Liu, L.; Liu, J.; Wu, W.; Zhou, H.; and Zhuang, B. 2024. Ptqd: Accurate post-training quantization for diffusion models. *Advances in Neural Information Processing Systems*, 36.

Heusel, M.; Ramsauer, H.; Unterthiner, T.; Nessler, B.; and Hochreiter, S. 2017. Gans trained by a two time-scale update rule converge to a local nash equilibrium. *Advances in neural information processing systems*, 30.

Ho, J.; Jain, A.; and Abbeel, P. 2020. Denoising diffusion probabilistic models. *Advances in Neural Information Processing Systems*, 33: 6840–6851.

Ho, J.; Saharia, C.; Chan, W.; Fleet, D. J.; Norouzi, M.; and Salimans, T. 2022. Cascaded diffusion models for high fidelity image generation. *The Journal of Machine Learning Research*, 23(1): 2249–2281.

Li, X.; Liu, Y.; Lian, L.; Yang, H.; Dong, Z.; Kang, D.; Zhang, S.; and Keutzer, K. 2023a. Q-diffusion: Quantizing diffusion models. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 17535–17545.

Li, Y.; Wang, H.; Jin, Q.; Hu, J.; Chemerys, P.; Fu, Y.; Wang, Y.; Tulyakov, S.; and Ren, J. 2024. Snapfusion: Text-to-image diffusion model on mobile devices within two seconds. *Advances in Neural Information Processing Systems*, 36.

Li, Z.; Xiao, J.; Yang, L.; and Gu, Q. 2023b. Repq-vit: Scale reparameterization for post-training quantization of vision transformers. In *ICCV*, 17227–17236.

Lin, J.; Tang, J.; Tang, H.; Yang, S.; Dang, X.; and Han, S. 2023. AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration. *arXiv preprint arXiv:2306.00978*.

Liu, Y.; Zhang, K.; Li, Y.; Yan, Z.; Gao, C.; Chen, R.; Yuan, Z.; Huang, Y.; Sun, H.; Gao, J.; et al. 2024. Sora: A Review on Background, Technology, Limitations, and Opportunities of Large Vision Models. *arXiv preprint arXiv:2402.17177*.

Liu, Z.; Lin, Y.; Cao, Y.; Hu, H.; Wei, Y.; Zhang, Z.; Lin, S.; and Guo, B. 2021. Swin transformer: Hierarchical vision transformer using shifted windows. In *ICCV*.

Martinez, J.; Shewakramani, J.; Liu, T. W.; Bârsan, I. A.; Zeng, W.; and Urtasun, R. 2021. Permute, quantize, and fine-tune: Efficient compression of neural networks. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 15699–15708.

Nash, C.; Menick, J.; Dieleman, S.; and Battaglia, P. W. 2021. Generating images with sparse representations. *arXiv preprint arXiv:2103.03841*.

Peebles, W.; and Xie, S. 2023. Scalable diffusion models with transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, 4195–4205.

Perez, E.; Strub, F.; De Vries, H.; Dumoulin, V.; and Courville, A. 2018. Film: Visual reasoning with a general conditioning layer. In *AAAI*.

Ramesh, A.; Dhariwal, P.; Nichol, A.; Chu, C.; and Chen, M. 2022. Hierarchical text-conditional image generation with clip latents, 2022. *URL https://arxiv. org/abs/2204.06125*, 7.

Rombach, R.; Blattmann, A.; Lorenz, D.; Esser, P.; and Ommer, B. 2022. High-resolution image synthesis with latent diffusion models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, 10684–10695.

Russakovsky, O.; Deng, J.; Su, H.; Krause, J.; Satheesh, S.; Ma, S.; Huang, Z.; Karpathy, A.; Khosla, A.; Bernstein, M.; et al. 2015. Imagenet large scale visual recognition challenge. *IJCV*, 115: 211–252.

Saharia, C.; Chan, W.; Saxena, S.; Li, L.; Whang, J.; Denton, E.; Ghasemipour, S. K. S.; Ayan, B. K.; Mahdavi, S. S.; Lopes, R. G.; et al. 2022. Photorealistic text-to-image diffusion models with deep language understanding. *arXiv preprint arXiv:2205.11487*.

Salimans, T.; Goodfellow, I.; Zaremba, W.; Cheung, V.; Radford, A.; and Chen, X. 2016. Improved techniques for training gans. *Advances in neural information processing systems*, 29.

Simonyan, K.; and Zisserman, A. 2014. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*.

Stock, P.; Joulin, A.; Gribonval, R.; Graham, B.; and Jégou, H. 2019. And the bit goes down: Revisiting the quantization of neural networks. *arXiv preprint arXiv:1907.05686*.

Touvron, H.; Cord, M.; Douze, M.; Massa, F.; Sablayrolles, A.; and Jégou, H. 2021. Training data-efficient image transformers and distillation through attention. In *ICML*.

Vaswani, A.; Shazeer, N.; Parmar, N.; Uszkoreit, J.; Jones, L.; Gomez, A. N.; Kaiser, L.; and Polosukhin, I. 2017. Attention is all you need. In *NeurIPS*.

Wang, H.; Shang, Y.; Yuan, Z.; Wu, J.; and Yan, Y. 2024. QuEST: Low-bit Diffusion Model Quantization via Efficient Selective Finetuning. *arXiv preprint arXiv:2402.03666*.

Xie, E.; Wang, W.; Yu, Z.; Anandkumar, A.; Alvarez, J. M.; and Luo, P. 2021. SegFormer: Simple and efficient design for semantic segmentation with transformers. *Advances in Neural Information Processing Systems*, 34: 12077–12090.

Yang, L.; Zhang, Z.; Song, Y.; Hong, S.; Xu, R.; Zhao, Y.; Zhang, W.; Cui, B.; and Yang, M.-H. 2023. Diffusion models: A comprehensive survey of methods and applications. *ACM Computing Surveys*, 56(4): 1–39.

Yuan, Z.; Xue, C.; Chen, Y.; Wu, Q.; and Sun, G. 2022. Ptq4vit: Post-training quantization for vision transformers with twin uniform quantization. In *ECCV*, 191–207.

Zhu, Z.; Wang, X.; Zhao, W.; Min, C.; Deng, N.; Dou, M.; Wang, Y.; Shi, B.; Wang, K.; Zhang, C.; et al. 2024. Is Sora a World Simulator? A Comprehensive Survey on General World Models and Beyond. *arXiv preprint arXiv:2405.03520*.

7 Appendix

7.1 Additional Results

As illustrated in Figures 5 and 6, we present additional results at 256×256 and 512×512 resolutions. Our VQ4DiT is capable of generating high-quality images even under extremely low-bit conditions. We also present the pseudo-code of our proposed VQ4DiT in Algorithm 1.

7.2 Deployment Setup

To improve inference speed, we implemented a CUDA vector quantization kernel for vector-vector multiplication between sub-vectors of quantized weights and sub-vectors of activations. Small-sized codebooks are loaded into shared memory to reduce bandwidth pressure. All computations are performed in FP32. As shown in Figure 6, when using the kernel, the inference time of the quantized model is reduced to approximately one-third of the original.

Algorithm 1: Our proposed VQ4DiT algorithm

Input: Full-precision weight W of each layer
Input: Sampling timesteps T, CFG scale
Input: Random conditional information $\mathbf{y} \in [1, 1000]$
Output: Codebook C and assignments A of each layer

Initialization of Codebooks and Candidate Assignment Sets:
Use K-Means algorithm to cluster W into the initial C and A based on equation 4
Create candidate assignment sets A_c and their ratios R based on equation 5
Zero-data and block-wise calibration:
for $\mathbf{y}$ **do**
  for $t = T$ to 1 **do**
    Generate calibration feature of each DiT block at t with W
    Generate quantized feature of each DiT block at t with $\widehat{W} = RC[A_c]$
    Calibrate and update C and R
  end for
  if mean$(R) < 10^{-4}$ **then**
    break
  end if
end for
Select optimal assignments A with largest R

Method	Resolution	Size (MB)	CUDA	Time
FP	256×256	2553.35	n/a	61s
3-bit VQ	256×256	241.14	no	63s
	256×256	241.14	yes	**22s**
2-bit VQ	256×256	162.08	no	63s
	256×256	162.08	yes	**20s**
FP	512×512	2553.35	n/a	249s
3-bit VQ	512×512	241.14	no	253s
	512×512	241.14	yes	**90s**
2-bit VQ	512×512	162.08	no	252s
	512×512	162.08	yes	**82s**

Table 6: Inference time(s) of DiT XL/2 on a NVIDIA A100 GPU. 'CUDA' denotes whether the CUDA vector quantization kernel is being used. During model inference, the sampling timesteps are set to 256, and the CFG scale is set to 1.5.

FP

3-bit

2-bit

Figure 5: Images generated by VQ4DiT with 3-bit and 2-bit quantization on ImageNet 256×256.

FP

3-bit

2-bit

Figure 6: Images generated by VQ4DiT with 3-bit and 2-bit quantization on ImageNet 512×512.